

CISC-814 (Summer 2026) Assignment 2: MLOps Pipeline

Detail	Value
Weight	25% of final grade
Type	Group (3 students, same teams as A1)
CLOs	3 (Monitoring strategies), 4 (DevOps/MLOps improvement analysis)
Part A due	Friday, June 5, 2026 (end of week 6)
Part B due	Friday, June 12, 2026 (end of week 7)

1 Overview

In this assignment you will evolve the sentiment analysis app from Assignment 1 by replacing the TextBlob model with Qwen2.5-1.5B, a small but capable large language model. You will build a complete MLOps pipeline covering experimentation, model serving, A/B testing, and production monitoring.

The assignment has two parts:

- Part A focuses on MLflow experimentation — running few-shot prompting experiments and quantization evaluation, tracking everything with MLflow, and registering models in the Model Registry.
- Part B focuses on production MLOps — serving the model with vLLM, A/B testing original vs. quantized models with a traffic router, drift monitoring, and deploying to k3d with an Argo Rollouts canary strategy and automated drift analysis.

2 Starter Code

The starter application is in `starter/ml-sentiment-app/`. It provides:

- `data/` - Place your evaluation datasets here
- `k8s/` - Manifest files for k3d deployment
- `monitoring/` - Scripts and files related to monitoring the served models and their environment
- `quantization/quantize_model.py` - Quantization script (using GPTQ)
- `serving/traffic_router.py` - FastAPI server skeleton
- `training/src/train.py` - Training script skeleton with TODOs for MLflow setup, few-shot integration, and quantization evaluation
- `training/src/experiments/few_shot.py` - Few-shot inference logic (complete)
- `training/src/evaluate.py` - Evaluation utilities (accuracy, F1, confusion matrix)
- `training/src/data_loader.py` - Dataset loading helpers
- `training/configs/experiment_config.yaml` - Experiment configuration
- `training/requirements.txt` - Dependencies (transformers, mlflow, gptqmodel, etc.)

These would be the essential steps needed to run the application on your machine (the assignment of course aims to get this to work in a vllm/k3d setup):

```
cd starter/ml-sentiment-app/training
```

```
# Install requirements (some warning messages are possible)
```

```
pip install -r requirements.txt --extra-index-url https://download.pytorch.org/whl/cu128
```

```
pip install -q vllm==0.18.0 evidently pandas prometheus-client
```

```
# Install quantization deps separately (needs --no-build-isolation)
```

```
pip install -q gptqmodel --no-build-isolation
```

```
# Generate sample training and evaluation datasets
python -c "from src.data_loader import create_sample_dataset; \
    create_sample_dataset('data/eval_sentiment.csv', n_samples=100); \
    create_sample_dataset('data/train_sentiment.csv', n_samples=200)"

# Run training (after completing TODOs)
python -m src.train
```

3 Infrastructure

All infrastructure from Assignment 1 remains available. This assignment adds MLflow and Evidently. See the infrastructure README and [tips section](#) for setup instructions.

Service	k3d In-Cluster Address	Host Address
Gitea	<code>http://cisc814-gitea:3000</code>	<code>http://localhost:3000</code>
Container Registry	<code>registry.local:5000</code>	<code>localhost:5001</code>
MLflow	<code>http://cisc814-mlflow:5000</code>	<code>http://localhost:5050</code>
Evidently	<code>http://cisc814-evidently:8000</code>	<code>http://localhost:8085</code>
Prometheus	see <code>manage-cluster.sh</code> info	-
Grafana	see <code>manage-cluster.sh</code> info	-

IMPORTANT:

- create a new git repository in your existing gitea setup to keep track of your history of commits on this assignment
- for Part B, be sure to create a new k3d cluster (see instructions below): do *not* remove the old cluster, since we still need it to mark your first assignment!

4 Part A: MLflow + Experimentation (40 points)

(Optionally) Due: Friday, June 5, 2026

Run few-shot experiments and quantization evaluation with Qwen2.5-1.5B, tracked in MLflow.

4.1 Tasks

0. Dependency Installation and Data Generation

- Install, in a Python venv, all training- and quantization-related Python dependencies mentioned [above](#). There will be warnings during the dependency installation, but you can check if the major functionality works as follows:

```
echo " Verifying key packages..."
python3 -c "import torch; print(f' PyTorch {torch.__version__}, CUDA available: {torch.cuda.is_available()}')"
python3 -c "import transformers; print(f' Transformers {transformers.__version__}')"
python3 -c "import mlflow; print(f' MLflow {mlflow.__version__}')"
python3 -c "import vllm; print(f' vLLM {vllm.__version__}')"

```

- Use the [same instructions](#) to create a training and an evaluation data set.

1. MLflow Setup (10 pts)

- Connect to your MLflow server (`http://localhost:5050`), and explore the user interface.
- Create an experiment with artifact proxying (specify `artifact_location="mlflow-artifacts:/"`).
- All runs for this experiment must log the following: parameters, metrics, and artifacts.
- Once you are happy with the setup, let's move to the next task, which will start populating experiment data.

2. Few-Shot Experiment Runs (12 pts)

- The few-shot inference logic in `training/src/experiments/few_shot.py` is already complete. Your task is to add MLflow logging calls so that each experiment run is tracked. The places where you need to add code are marked with TODO comments in:

- `setup_mlflow` in `training/src/train.py`

- main in training/src/train.py
- run_few_shot in training/src/experiments/few_shot.py

Follow the TODO comments for hints on what to log (parameters, metrics, artifacts). Normally, one would invoke `mlflow.transformers.log_model` to store the model weights in MLflow. However, due to the limited VM resources available for the assignment, we instead store the Hugging Face model name as a parameter called `vllm_model_path`. In Part B, we will retrieve the actual model weights using this metadata.

- Once finished, run the few-shot experiment runs, and observe the MLflow user interface to analyze the logged metrics of all runs within the MLflow experiment.
- Make screenshots of tables, figures or other important artifacts in the user interface for your few-shot runs. Include those in the final report for assignment 2.

3. Quantization Experiment Runs (10 pts)

- We are going to derive a lighter-weight variant of Qwen2.5-1.5B by quantization. The quantization logic in `quantization/quantize_model.py` is provided; your task is to add MLflow logging to the `evaluate_quantized_model` function so that post-quantization metrics (accuracy, F1, latency, model size) are tracked. See the TODO comments in that function for details.
- As in task 2, instead of calling `mlflow.transformers.log_model` (which would store the large model weights in MLflow), log the local file path of the quantized model as a `vllm_model_path` parameter. Since quantized models exist only on disk (not on Hugging Face), this path will let Part B locate and serve them.
- Read through functions `quantize_gptq` and `evaluate_quantized_model`, then explain in your own words what these are doing. Include this explanation in the final report for assignment 2.
- Once finished, run the quantization experiment run using the following suggested parameters (adapt the paths and names to your context):

```
python3 quantize_model.py \
  --model Qwen/Qwen2.5-1.5B \
  --output ./qwen2.5-1.5b-gptq-4bit \
  --method gptq \
  --bits 4 \
  --num-samples 48 \
  --evaluate \
  --eval-dataset "path_to_eval_sentiment.csv" \
  --mlflow-uri http://localhost:5050 \
  --experiment-name "the_experiment_name"
```

Hint:

- Quantization is slow (typically 45 minutes to 2 hours). Monitor GPU usage with `nvidia-smi` to confirm the process is still active.

4. Model Registry (8 pts)

- Open `training/src/registration.py` and implement the `register_model` function. Given an experiment type (e.g., `few_shot` or `quantization_eval`), it should find the best run by F1 score, create a model version in the MLflow Model Registry, and assign the specified alias. Based on tasks 2 and 3, you will register the best few-shot model as `champion` and the best quantized model as `challenger`.
- Once finished, run the resulting script twice, as such (making sure to adjust any paths or names to your context):

```
python3 -m src.registration \
  --tracking-uri http://localhost:5050 \
  --experiment-name "$EXPERIMENT_NAME" \
  --experiment-type few_shot \
  --alias champion
```

```
python3 -m src.registration \
  --tracking-uri http://localhost:5050 \
  --experiment-name "$EXPERIMENT_NAME" \
  --experiment-type quantization_eval \
  --alias challenger
```

- Finally, visit the MLflow user interface, and double-check that the aliases have been assigned correctly. Include a screenshot in your final report for assignment 2.

Hint:

- Make sure that both model aliases are registered under the same MLflow experiment.

4.2 Grading Table - Part A

Component	Points
MLflow setup + experiment tracking (URI, experiment, proxied artifacts)	10
Few-shot experiments with MLflow (1/3/5-shot, params/metrics/artifacts)	12
Quantization eval + MLflow logging (accuracy, F1, latency, model size)	10
Model registry (champion + challenger aliases)	8
Total	40

5 Part B: Production MLOps (60 points)

Due: Friday, June 12, 2026

Part B builds on Part A: you will serve the models you registered in MLflow, A/B test them, monitor for drift, and deploy to k3d with automated canary rollouts. Make sure you have completed Part A (champion and challenger models registered in MLflow) before starting.

Throughout Part B, we refer to the champion model (original fp16) as V1 and the challenger model (quantized GPTQ) as V2.

5.1 Tasks

0. Install Python dependencies

- All dependencies should still be available in the virtual environment you used for Part A, you can check this using the following command:

```
python3 -c "import vllm; print(f' vLLM {vllm.__version__}')
```

1. Model Serving with vLLM (12 pts)

- Use a short, inline Python script (using `python3 -c "[script code]"`) to access the "champion" (referred to as "V1" below) and "challenger" ("V2" below) models, and obtain their physical location via the value of their `vllm_model_path` version tags. Normally, one would actually store the real model weights in MLflow, then download those files. Due to resource limitations, we only stored the actual Hugging Face name (few shot) or path (quantization), so we can directly access the previously downloaded files locally (the Hugging Face model will still be in the local cache folder).
- Start vLLM locally for both the V1 (original fp16) and V2 (quantized GPTQ) models, using the Hugging Face name/path we just obtained from MLflow. Use a command similar to the following (adjusting paths/names):

```
python3 -m vllm.entrypoints.openai.api_server \
  --model "$CHAMPION_PATH" \
  --port 8001 \
  --max-model-len 512 \
  --gpu-memory-utilization 0.45 \
  --enforce-eager \
```

- Use `monitoring/send_predictions.py` to send some example queries to both servers directly to verify whether both respond correctly to sentiment queries. Make screenshots of these queries and responses, and also save the vLLM servers' log files.

2. A/B Testing with Traffic Router (12 pts)

- Instead of directly connecting to V1 or V2, we are interested in understanding whether the quantized model (V2) is good enough to be release, before doing an actual release of the new model. In other words, we will do an A/B test in which part of the user traffic will be routed to V2, while the majority will still be sent to the safer older model. Here, we will use a FastAPI traffic router with 80/20 split (V1/V2) to show the mechanics of A/B testing involving models.
- The initial, incomplete code of the traffic router (inspired by the `inference.py` script of assignment 1) can be found in `serving/traffic_router.py`. Complete the code of the following two endpoints using the provide code comments:

- predict
- stats
- Use this command to run the router process: `uvicorn traffic_router:app --host 0.0.0.0 --port 8000`
- Send 250+ test predictions through the router, using query code similar to this:

```
TEST_TEXTS=(
    "I absolutely love this product, it is amazing!"
    "Terrible service, worst experience ever."
    "The package arrived on time."
    "Great quality and fantastic value for money!"
    "Completely broken, waste of money."
    "It works as expected, nothing special."
    "Outstanding results, exceeded all expectations!"
    "Poor quality, falls apart quickly."
    "Standard delivery, standard product."
    "Incredible improvement over the previous version!"
)

echo " Sending 250 predictions through the router..."
for i in $(seq 1 250); do
    idx=$(( (i - 1) % ${#TEST_TEXTS[@]} ))
    text="${TEST_TEXTS[$idx]}"
    result=$(curl -s -X POST http://localhost:8000/predict \
        -H 'Content-Type: application/json' \
        -d "{\"text\": \"$text\"}")
    if [ $((i % 10)) -eq 0 ]; then
        echo "    [$i/250] $result"
    fi
done

curl -s http://localhost:8000/stats
```

Hints:

- After running the test requests, also check the `.jsonl` log file generated by the router

3. Drift Monitoring between V1 and V2 (10 pts)

- Use the `grep` bash command to split the prediction logs obtained in the previous task by model version ("v1" vs "v2")
- Then run Evidently drift detection comparing the V1 (reference) and V2 (current) models, using `monitoring/drift_detector.py` to generate a drift report:

```
python3 drift_detector.py \
    --reference v1_predictions.jsonl \
    --current v2_predictions.jsonl \
    --output drift_report.html \
    --push
```

- Based on the generated drift report, analyze whether quantization caused distributional shift in the data: does latency drift between the fp16 and GPTQ models? does label distribution shift? is the drift meaningful or just noise from a small sample?
- Write up your observations for the report.

4. k3d Deployment + Argo Rollouts Canary (16 pts)

- To deploy the models in a k3d environment, we will provision containers based on the default `vllm/vllm-openai:v0.18.0` image. This base vLLM image will be pushed to the local registry, then, upon instantiation of containers, we will mount model weights as volumes (production-standard pattern). In other words, all containers will be identical, the sole difference between V1 and V2 containers will be the specific model file (volume) path they will use.
 - First, tag the Hugging Face model `vllm/vllm-openai:v0.18.0` as `localhost:5001/vllm-base:latest`, and push it to the registry.

- Then, build the Docker image defined by `monitoring/Dockerfile.analysis` (which packages `drift_detector.py` and `send_predictions.py` into an analysis job container), tagging it as `localhost:5001/sentiment-analysis-job:latest`, and push it to the registry as well.
- Check the registry's contents: `curl -s http://localhost:5001/v2/_catalog`
- Now create a new (!) k3d cluster with a name of your choice, for instance `cluster2` (please replace this name by your cluster's name in the below commands). Make sure to use the following commands and to update any paths (except the `/mnt/...` ones):

```
echo " Cleaning up Docker build cache to free disk space..."
docker builder prune -f --all 2>/dev/null || true

cd "../../../../../infrastructure/k3d"
bash manage-cluster.sh create --gpu \
  --volume "$HOME/.cache/huggingface:/mnt/hf-cache" \
  --volume "quantization/qwen2.5-1.5b-gptq-4bit:/mnt/models/v2-quantized" \
  cluster2
kubectl config use-context k3d-cisc814-cluster2
```

- Before we can deploy any container, we need to import the custom Docker image into k3d:

```
k3d image import \
  "localhost:5001/sentiment-analysis-job:latest" \
  -c cisc814-cluster2
```

- We now need to create the k3d manifests required for deploying V1 and V2 models using Argo Rollouts. Start by looking at the `k8s/configmap.yaml`, `k8s/service.yaml`, `k8s/pushgateway-servicemonitor.yaml` and `k8s/grafana-dashboard.yaml` manifests, which are complete. Note that we will not use the traffic router here to split traffic, to simplify things, but instead control the pod ratio via Argo Rollouts. Once you are ready, complete the following files, for which only a skeleton (with detailed comments/hints) is provided:

- `k8s/rollout.yaml`
- `k8s/analysis-template.yaml`

The rollout manifest describes how a new model version should be deployed using canary steps. The analysis template defines an automated gate that monitors whether the canary is safe to promote: it runs the `sentiment-analysis-job` container to send test predictions to both V1 and V2, computes a drift share (fraction of features with detected distributional shift), pushes that score to Prometheus, and then a `drift-score` metric check queries Prometheus to decide pass/fail based on a threshold.

- Once done, apply all manifests, and watch the initial rollout of V1 in the Argo Rollouts dashboard and the Grafana dashboard that was deployed.
- Finally, we will trigger a canary rollout (V1 → V2), as long as V2 does not drift too much compared to V1. Use the following commands for this (they override the model file path and model name, causing a new rollout):

```
kubectl patch rollout sentiment-model --type='json' -p='[
  {"op": "replace", "path": "/spec/template/spec/containers/0/args/1", "value": "/mnt/models/v2-quantized"},
  {"op": "replace", "path": "/spec/template/spec/containers/0/env/0/value", "value": "v2-quantized"}
]'
```

- Observe the automated promotion/rollback in the Argo Rollouts dashboard, and also watch the Grafana dashboard. Make screenshots of both.

5. Benchmarking Report + Presentation (10 pts)

- Your report should contain two architectural diagrams showing, respectively, the serving architecture used in Part A and the one used in Part B. Briefly explain each component, the files it contains and also indicate the specific assignment Part/task it was used in.
- Also include the various screenshots, experimental results and observations requested in Parts A and B.
- Furthermore, create a 10-minute, step-by-step live demo of the outcomes of your Part A experiments with quantization in MLflow and of how to deploy a new canary model in the Part B k3d setup (while guarding for drift). The video again should be presented by one team member (whose face is shown on top of the video). Upload the video to OneDrive, Google Drive or as an unlisted video on YouTube, then include the URL in your report. Check permissions of this video so the instructors can access it.
- Submit your report on onQ in the form of a pdf file. Don't forget to include the names of all team members, as well as the IP address and name of your team's VM (so we can access and evaluate your manifests and other files).

5.2 Grading Table - Part B

Component	Points
Model serving with vLLM (local V1 + V2, MLflow registry paths)	12
A/B testing (traffic router, 80/20 split, /predict, /stats)	12
Drift monitoring - local (Evidently V1 vs V2 comparison)	10
K3d + Argo Rollouts Canary (GPU deploy, AnalysisRun, promotion)	16
Report + 10-min presentation	10
Total	60

6 Deliverables Summary

Part	What to Submit	Where
A	MLflow experiments visible on course server, training scripts	Gitea repo
B	Serving + A/B + monitoring code, K8s manifests, report PDF, presentation	Gitea repo + OnQ

7 Repository Structure (Expected)

```
team-<name>-mllops/
├── training/
│   ├── src/
│   │   ├── train.py
│   │   ├── registration.py
│   │   ├── evaluate.py
│   │   ├── data_loader.py
│   │   └── experiments/
│   │       └── few_shot.py
│   ├── configs/
│   │   └── experiment_config.yaml
│   └── requirements.txt
├── quantization/
│   └── quantize_model.py
├── serving/
│   ├── traffic_router.py
│   ├── Dockerfile
│   └── requirements.txt
├── k8s/
│   ├── rollout.yaml
│   ├── analysis-template.yaml
│   ├── service.yaml
│   ├── configmap.yaml
│   ├── pushgateway-servicemonitor.yaml
│   └── grafana-dashboard.yaml
├── monitoring/
│   ├── drift_detector.py
│   ├── send_predictions.py
│   ├── analysis_entrypoint.sh
│   ├── Dockerfile.analysis
│   └── evidently_config.yaml
└── README.md
```

8 Resources

- [Qwen2.5 on Hugging Face](#)
- [MLflow documentation](#)

- [vLLM documentation](#)
- [Evidently AI](#)
- [GPTQModel](#) (successor to AutoGPTQ)
- [Argo Rollouts - Canary](#)
- [Argo Rollouts - Analysis](#)

9 Tips

- Start Part A early, since model downloads and (especially) quantization take a lot of time.
- Note that the Python dependency installation might yield some warnings (perhaps some errors), please ignore them, as long as the test commands in task 0 of Part A succeed.
- Use the sample dataset generator to test your pipeline before using real data.
- Monitor GPU memory carefully: Qwen2.5-1.5B needs ~3 GB in fp16, ~1 GB in GPTQ 4-bit. Before deploying to k3d, kill any leftover local vLLM servers to free VRAM:

```
kill -f "vllm.entrypoints" 2>/dev/null; sleep 2; nvidia-smi --query-gpu=index,memory.free --format=csv
```

- For Part B, get vLLM to work locally (in your VM) first, then add the traffic router, only then move to k3d.
- Monitor your Argo Rollouts in real time with:

```
kubectl argo rollouts get rollout sentiment-model --watch
```

For the web dashboard, start it on your server VM and use SSH port forwarding to access it locally on your laptop:

```
# On your local machine (add to your SSH command):
```

```
ssh -L 3100:localhost:3100 localuser@<server>
```

```
# On the server:
```

```
kubectl argo rollouts dashboard --port 3100 &
```

Then open <http://localhost:3100> in your laptop's browser.

- MLflow artifact storage: When creating an experiment, ensure that that artifacts are proxied correctly by using `MlflowClient.create_experiment(name, artifact_location="mlflow-artifacts:/")`. Using `mlflow.set_experiment()` alone does not set the artifact location, which can cause artifact uploads to fail or go to the wrong place.
- vLLM + MLflow: vLLM does not natively read from MLflow. Query the MLflow API for the artifact path of `champion/challenger` aliases, then pass those paths as `--model` args to vLLM.

10 Academic Integrity

This is a group assignment. All team members must contribute meaningfully. You may consult documentation and online resources, but all pipeline configuration and manifests must be your own work. Include a section in your report that lists each member's contributions.

You are *allowed* to use AI for this assignment, as long as you explain clearly in your report:

- how you selected the specific AI model
- what tasks were assigned to the model
- how your group ensured the quality of the model's output

11 Peer Evaluation

Each team member will submit a separate, confidential peer evaluation email to the Queen's TA after submission of Part B. In this email, you should provide an objective and frank account of each team member's contributions to the assignment, including your own contributions. Include both positive and negative aspects of each team member's participation. This feedback is essential for us to understand whether every team member collaborated as expected, or if someone tried to get easy marks. Grades may be adjusted based on peer feedback.

Concretely:

- within 24 hours after submission of your group's assignment on onQ, send your individual peer review email to ahmed.hereiz@queensu.ca
- the subject of the email should be "[CISC-814] Peer review feedback for assignment 2 (group XYZ)" where you replace XYZ by your team's number (see onQ)
- write one paragraph per group member, including yourself, listing positive and negative aspects about the member's participation
- if we do not receive a peer review email within 24 hours, you may lose marks for this assignment, so please do not forget this email